

GMX · SECURITY REVIEW

GMX

Buyback Updates.

A security review of Buyback Updates.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
GMX	Buyback Updates	Guardian	September 23 – October 9, 2024	October 16, 2024

§ 00

Contents.

44 pages — overview, confidence ranking, scope, 22 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 20 findings	P. 09
	Remediation Review · 2 findings	P. 41
§ 05	Disclaimer	P. 43
§ 06	About Guardian	P. 44

§ 01

Overview.

Guardian reviewed Buyback Updates across 2 review rounds. The final report includes 1 Critical, 3 High, 3 Medium, 15 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	gmx-io/gmx-contracts
CLIENT	METHODOLOGY
GMX	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Buyback Updates	c36f2e0cce70
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	922f015a1faf
AUDIT WINDOW	
September 23 – October 9, 2024	
FINAL REPORT	
October 16, 2024	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	1	0	0	0	1
■ High	3	0	0	0	3
■ Medium	3	0	2	0	1
■ Low	15	0	9	0	6
■ Informational	0	0	0	0	0
All severities	22	0	11	0	11

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Buyback Updates in `gmx-io/gmx-contracts`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Necessary Handler Statuses Are Not Set	DoS	CRITICAL	RESOLVED	p. 9
H-01	Old Reward Router Cannot Be Disabled	Logical Error	HIGH	RESOLVED	p. 11
H-02	Incorrect Vesting Validation On Account Transfer	Logical Error	HIGH	RESOLVED	p. 12
H-03	Account Transfers Abused To Invalidate Vesting	Logical Error	HIGH	RESOLVED	p. 13
M-01	Use Of DelegateCall Within A For Loop In A Payable Multicall	Logical Error	MEDIUM	RESOLVED	p. 17
M-02	ExternalHandler Tokens Could Be Drained	Logical Error	MEDIUM	ACKNOWLEDGED	p. 19
M-03	esGMX Claimed To Any Address	Logical Error	MEDIUM	ACKNOWLEDGED	p. 22
L-01	Users Cannot Unstake Until Re-staked	Documentation	LOW	RESOLVED	p. 23
L-02	RewardRouterV2 Contract Missing GOV_TOKEN_CONTROLLER Role	DoS	LOW	ACKNOWLEDGED	p. 24
L-03	_restakeBnGmx Ignores RewardRouterV2 maxAllowedBnGmxAmount	Logical Error	LOW	RESOLVED	p. 26
L-04	FeeGmxTracker Should Only Allow To Stake sbeGMX	Configuration	LOW	ACKNOWLEDGED	p. 28

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-05	Multiple Operations Require An Unintended GMX/bnGMX Approval	Configuration	LOW	RESOLVED	p. 29
L-06	Missing Initialized Values	Warning	LOW	ACKNOWLEDGED	p. 32
L-07	Lacking Zero Address Checks	Validation	LOW	ACKNOWLEDGED	p. 33
L-08	VesterCap May Not Function Correctly	Warning	LOW	ACKNOWLEDGED	p. 34
L-09	VesterCap maxBoostBasisPoints Inconsistency	Warning	LOW	ACKNOWLEDGED	p. 35
L-10	Missing nonReentrant Modifier	Reentrancy	LOW	RESOLVED	p. 36
L-11	validateReceiver Is Missing Checks On The extendedGmxTracker	Logical Error	LOW	RESOLVED	p. 37
L-12	Some Accounts Cannot Be Re-Staked	Warning	LOW	ACKNOWLEDGED	p. 39
L-13	Some Accounts May Need To Be ReReStaked	Warning	LOW	ACKNOWLEDGED	p. 40

REMEDIATION REVIEW

L-01	Private Staking Mode	Warning	LOW	ACKNOWLEDGED	p. 41
L-02	Unused Variable	Superfluous Code	LOW	RESOLVED	p. 42

CRITICAL **C-01**

RESOLVED

Necessary Handler Statuses Are Not Set

CATEGORY	DoS
LOCATION	BuybackMigrator.sol
REVIEW	Main Review

DESCRIPTION

In the `BuybackMigrator` contract the `_toggleRewardRouter` function does not set the `extendedGmxTracker` contract as a handler for the `bonusGmxTracker` and `bnGmx` tokens.

As a result the `stakeForAccount` function requires approval from the user as it relies on `safeTransferFrom` for the staked token. Thus the `RewardRouterV2.batchRestakeForAccounts` function does not work unless users explicitly approve the new `extendedGmxTracker` contract, which is not the expected behavior.

Additionally, the `feeGmxTracker` needs to be set as a handler for the `extendedGmxTracker` token to be able to transfer it in.

The current impact is that any `stakeGmx()` call would revert and that users would be required to perform multiple approvals upon staking/compounding...

Finally, the `GlpManager` should set as handler the new `RewardRouterV2` contract otherwise, calls to the `mintAndStakeGlp`, `mintAndStakeGlpETH`, `unstakeAndRedeemGlp` and `unstakeAndRedeemGlpETH` functions will revert.

RECOMMENDATION

Enable the correct handler statuses in the `_toggleRewardsRouter` function with the following:

```
if(isEnabled){  
  IHandlerTarget(bonusGmxTracker).setHandler(extendedGmxTracker, true);  
  IHandlerTarget(extendedGmxTracker).setHandler(feeGmxTracker, true);  
  IHandlerTarget(bnGmx).setHandler(extendedGmxTracker, true);  
}
```

On the other hand, consider setting the new `RewardRouterV2` contract as a handler for the `GlpManager` contract.

C-01 Necessary Handler Statuses Are Not Set

CONTINUED

— RESOLUTION

Status: Resolved.

HIGH **H-01**

RESOLVED

Old Reward Router Cannot Be Disabled

CATEGORY Logical Error
LOCATION BuybackMigrator.sol
REVIEW Main Review

DESCRIPTION

In the `disableOldRewardRouter` function there is no request for the gov permission of the `extendedGmxTracker`.

As a result when the `afterGovGranted` callback is invoked, it will revert as the `BuybackMigrator` address has not been granted governorship over the `extendedGmxTracker`.

Currently in the test setup the `extendedGmxTracker` governor is initially assigned as the timelock:
<https://github.com/gmx-io/gmx-contracts/pull/149/files#diff-2ceafaae01d44042116d49f1333f49a9a808d6378b176ef9d72ac111968592d0R278>

RECOMMENDATION

Request the gov role for the `extendedGmxTracker` in the list of targets. Or otherwise ensure that the `BuybackMigrator` contract is the governor of the `extendedGmxTracker` when the old reward router is disabled.

RESOLUTION

Status: Resolved.

HIGH **H-02**

RESOLVED

Incorrect Vesting Validation On Account Transfer

CATEGORY Logical Error
LOCATION RewardRouterV2.sol: 681
REVIEW Main Review

DESCRIPTION

In the `acceptTransfer` function the `_validateNotVesting` internal function is called to validate that the sender does not currently have any vested tokens.

The original `acceptTransfer` validation validated that neither the `gmxVester` nor the `glpVester` could have vested tokens:

```
require(IERC20(gmxVester).balanceOf(_sender) == 0, "sender has vested tokens");  
require(IERC20(glpVester).balanceOf(_sender) == 0, "sender has vested tokens");
```

However the `_validateNotVesting` function allows the sender to have one of the `gmxVester` or `glpVester` with vesting tokens:

```
require(IERC20(gmxVester).balanceOf(_sender) == 0 || IERC20(glpVester).balanceOf(_sender) ==  
0, "sender has vested tokens");
```

Thus the sender can have a nonzero amount of vesting tokens in the `gmxVester` OR the `glpVester` at the time of transfer. This can invalidate subsequent vesting logic as the average staked amounts used to determine the amount of paired tokens necessary for the sender's vest are changed on account transfer with the `transferStakeValues` function.

RECOMMENDATION

Change the validation in the `_validateNotVesting` function to:

```
require(IERC20(gmxVester).balanceOf(_sender) == 0 && IERC20(glpVester).balanceOf(_sender) ==  
0, "sender has vested tokens");
```

RESOLUTION

Status: Resolved.

HIGH **H-03**

RESOLVED

Account Transfers Abused To Invalidate Vesting

CATEGORY Logical Error
LOCATION RewardRouterV2.sol: 390, Vester.sol: 133
REVIEW Main Review

DESCRIPTION

The `Vester` contract implements the function `transferStakeValues`:

```
function transferStakeValues(address _sender, address _receiver) external override
nonReentrant {
    _validateHandler();
    transferredAverageStakedAmounts[_receiver] = getCombinedAverageStakedAmount(_sender);
    transferredAverageStakedAmounts[_sender] = 0;
    uint256 transferredCumulativeReward = transferredCumulativeRewards[_sender];
    uint256 cumulativeReward = hasRewardTracker() ?
    IRewardTracker(rewardTracker).cumulativeRewards(_sender) : 0;
    transferredCumulativeRewards[_receiver] = transferredCumulativeReward.add(cumulativeReward);
    cumulativeRewardDeductions[_sender] = cumulativeReward;
    transferredCumulativeRewards[_sender] = 0;
    bonusRewards[_receiver] = bonusRewards[_sender];
    bonusRewards[_sender] = 0;
}
```

This function is used every time the `acceptTransfer` function is called in the `RewardRouterV2`:

```
IVester(gmxVester).transferStakeValues(_sender, receiver);
IVester(glpVester).transferStakeValues(_sender, receiver);
```

As defined in the [GMX docs](https://gmx-docs.io/docs/tokenomics/rewards#escrowed-gmx), this is the function that implements the full account transfer flow which transfers all the GMX, esGMX, GLP, Multiplier Points and voting power to a new account.

H-03 Account Transfers Abused To Invalidate Vesting

CONTINUED

On the other hand, the `Vester` contract is also responsible for the vesting of esGMX tokens. esGMX represents vested rewards and the `Vester` contract allows users to gradually convert these esGMX tokens into fully transferable GMX tokens over a set period. This contract enforces a limit on the amount of esGMX tokens that can be locked for GMX:

```
function getMaxVestableAmount(address _account) public override view returns (uint256) {
    uint256 transferredCumulativeReward = transferredCumulativeRewards[_account];
    uint256 bonusReward = bonusRewards[_account];
    uint256 maxVestableAmount = transferredCumulativeReward.add(bonusReward);
    if (hasRewardTracker()) {
        uint256 cumulativeReward = IRewardTracker(rewardTracker).cumulativeRewards(_account);
        maxVestableAmount = maxVestableAmount.add(cumulativeReward);
    }
    uint256 cumulativeRewardDeduction = cumulativeRewardDeductions[_account];
    if (maxVestableAmount < cumulativeRewardDeduction) {
        return 0;
    }
    return maxVestableAmount.sub(cumulativeRewardDeduction);
}
```

The `getMaxVestableAmount` function determines how this limit is calculated. As it can be seen in the code above, the higher the `transferredCumulativeReward` the higher the amount of esGMX that can be locked.

Therefore, the following exploit could be possible:

1. Alice stakes 100 GMX through the `RewardRouterV2`.
2. A week later, Alice compounds her rewards. This increases her `StakedGMXTracker` `cumulativeRewards` to 87626020444780629.
3. Alice now unstakes the 100 GMX through the `RewardRouterV2`.
4. Alice has 10000 GMX tokens available to stake.
5. Alice creates 100 different accounts. For each of those accounts Alice does a full account transfer from her initial account to those 100 accounts. This is done by calling `RewardRouterV2.signalTransfer(destAcc)` from Alice's account and then calling `RewardRouterV2.acceptTransfer(AliceAcc)` from each of the different accounts created.
6. With each of these `acceptTransfer` calls the receiver account gets its `MaxVestableAmount` set to 87626020444780629. This is the source of the exploit as the `cumulativeReward` is always assigned to the receiver account but never decremented for the sender as can be seen in the following line of the `transferStakeValues` function: `uint256 cumulativeReward = hasRewardTracker() ? IRewardTracker(rewardTracker).cumulativeRewards(_sender) : 0;`
7. Alice now sends 100 GMX tokens to each of her 100 created accounts (100 x 100 = 10000)

H-03 Account Transfers Abused To Invalidate Vesting

CONTINUED

and stakes them through the `RewardRouterV2`.

8. Alice is now able to deposit an additional 87626020444780629 esGMX tokens per account in the `Vester` contract, for a total of $87626020444780629 \times 100 = 8762602044478062900$ tokens.

Furthermore in the `transferStakeValues` function, the `transferredAverageStakedAmounts` is set to zero for the sender account, but the `rewardTracker.cumulativeRewards(sender)` is not reset. As a result subsequent receivers will receive the full `transferredCumulativeRewards` amount, while not receiving the `transferredAverageStakedAmounts` that the sender was originally assigned.

Thus it is possible for the receiver to only be required to stake 0 paired sbfGMX tokens to vest their esGMX. This is because the `getPairAmount` function computes the paired amount as a ratio of the `combinedAverageStakedAmount` compared to the `maxVestableAmount`.

In the scenario where the sending account holds their entire `averageStakedAmount` in the `transferredAverageStakedAmounts` mapping, the numerator of the `getPairAmount` ratio is assigned to zero after the first account transfer out of the sender account. However the cumulative rewards are not reset as described above.

— RECOMMENDATION

Consider implementing the following logic for the `transferStakeValues` function:

```
function transferStakeValues(address _sender, address _receiver) external override
nonReentrant {
    _validateHandler();
    transferredAverageStakedAmounts[_receiver] = getCombinedAverageStakedAmount(_sender);
    transferredAverageStakedAmounts[_sender] = 0;
    uint256 transferredCumulativeReward = transferredCumulativeRewards[_sender];
    - uint256 cumulativeReward = hasRewardTracker() ?
IRewardTracker(rewardTracker).cumulativeRewards(_sender) : 0;
    + uint256 cumulativeReward = hasRewardTracker() ?
IRewardTracker(rewardTracker).cumulativeRewards(_sender) - cumulativeRewardDeductions[_sender]
: 0;
    transferredCumulativeRewards[_receiver] = transferredCumulativeReward.add(cumulativeReward);
    - cumulativeRewardDeductions[_sender] = cumulativeReward;
    + cumulativeRewardDeductions[_sender] += cumulativeReward;
    transferredCumulativeRewards[_sender] = 0;
    bonusRewards[_receiver] = bonusRewards[_sender];
    bonusRewards[_sender] = 0;
}
```

H-03 Account Transfers Abused To Invalidate Vesting

CONTINUED

PROOF OF CONCEPT https://github.com/r0bert-ethack/Guardian_GMX_23092024/blob/main/test/Deploy.sol#L1171

— RESOLUTION

Status: Resolved.

MEDIUM **M-01**

RESOLVED

Use Of DelegateCall Within A For Loop In A Payable Multicall

CATEGORY Logical Error
LOCATION RewardRouterV2.sol: 170
REVIEW Main Review

DESCRIPTION

The `RewardRouterV2` contract provides a `multicall()` function designed to batch multiple function calls into a single transaction. This function is declared as payable, suggesting it can accept native assets and forward them to the called functions:

```
function multicall(bytes[] memory data) external payable returns (bytes[] memory results) {  
    results = new bytes[](data.length);  
    for (uint256 i; i < data.length; i++) {  
        (bool success, bytes memory result) = address(this).delegatecall(data[i]);  
        require(success, "call failed");  
        results[i] = result;  
    }  
    return results;  
}
```

However, the `multicall()` function uses `delegatecall` to execute the provided calls. As a result, when users attempt to call the `mintAndStakeGlpETH()` function via `multicall()` the call forwards the full `msg.value` each time, thus effectively duplicating the `msg.value` provided.

In many cases this will simply cause the subsequent non-payable function calls to revert or cause subsequent payable function calls to revert as no more Ether is available in the `RewardRouterV2` contract.

RECOMMENDATION

Consider updating the `multicall()` function to correctly forward the `msg.value` to the called functions. A possible update could be:

```
function multicall(bytes[] memory data, uint256[] memory msgValues) external payable returns  
(bytes[] memory results) {
```

M-01 Use Of DelegateCall Within A For Loop In A Payable Multicall

CONTINUED

```
require(data.length == msgValues.length, "Diff array lengths");
results = new bytes[](data.length);
uint256 msgValue = msg.value;
uint256 totalMsgValue;
for(uint256 i; i < msgValues.length; ++i){
totalMsgValue += msgValues[i];
}
require(totalMsgValue == msgValue, "Diff msgValues");
for (uint256 i; i < data.length; i++) {
// Determine the amount of Ether to send with each call if necessary.
// For simplicity, this example sends all available Ether with each call.
// In a real-world scenario, you might need to split 'msg.value' among calls or accept an
array of values.
(bool success, bytes memory result) = address(this).delegatecall{value: msgValues[i]}
(data[i]);
require(success, "call failed");
results[i] = result;
}
return results;
}
```

Otherwise consider making the `multicall` function non-payable and accept the fact that payable functions may not be batched.

— RESOLUTION

Status: Resolved.

MEDIUM **M-02**

ACKNOWLEDGED

ExternalHandler Tokens Could Be Drained

CATEGORY Logical Error
LOCATION RewardRouterV2.sol: 422, ExternalHandler.sol
REVIEW Main Review

DESCRIPTION

The `RewardRouterV2` contract implements the function `makeExternalCalls()`:

```
function makeExternalCalls(  
    address[] memory externalCallTargets,  
    bytes[] memory externalCallDataList,  
    address[] memory refundTokens,  
    address[] memory refundReceivers  
) external {  
    IExternalHandler(externalHandler).makeExternalCalls(  
        externalCallTargets,  
        externalCallDataList,  
        refundTokens,  
        refundReceivers  
    );  
}
```

This function interacts with an `externalHandler` contract:

```
contract ExternalHandler is IExternalHandler, ReentrancyGuard {  
    using Address for address;  
    using SafeERC20 for IERC20;  
    // @notice refundTokens should be unique, this is because the refund loop  
    // sends the full refund token balance on each iteration, so if there are  
    // duplicate refund token addresses, then only the first refundReceiver  
    // for that token would receive the tokens  
    function makeExternalCalls(  
        address[] memory targets,  
        bytes[] memory dataList,  
        address[] memory refundTokens,  
        address[] memory refundReceivers  
    ) external nonReentrant {
```

M-02 ExternalHandler Tokens Could Be Drained

CONTINUED

```
if (targets.length != dataList.length) {
    revert Errors.InvalidExternalCallInput(targets.length, dataList.length);
}
if (refundTokens.length != refundReceivers.length) {
    revert Errors.InvalidExternalReceiversInput(refundTokens.length, refundReceivers.length);
}
for (uint256 i; i < targets.length; i++) {
    _makeExternalCall(
        targets[i],
        dataList[i]
    );
}
for (uint256 i; i < refundTokens.length; i++) {
    IERC20 refundToken = IERC20(refundTokens[i]);
    uint256 balance = refundToken.balanceOf(address(this));
    if (balance > 0) {
        refundToken.safeTransfer(refundReceivers[i], balance);
    }
}
}
```

This `ExternalHandler` contract is supposed to execute some external calls and interactions with the `RewardRouterV2`, receive tokens/rewards and then refund those to the receivers. However, the following exploit could be possible:

1. Alice calls the `ExternalHandler.makeExternalCalls()` and max. approves her smart contract with multiple tokens addresses on behalf of the `ExternalHandler` contract.
2. Bob, afterwards, calls `ExternalHandler.makeExternalCalls()` batching multiple external calls. During the first external call the `ExternalHandler` contract received some USDT and GMX tokens. In the second external call, Alice's smart contract receive some native assets, receiving the control.
3. Alice's smart contract, which is max. approved by the `ExternalHandler` contract simply drains all the tokens.
4. Bob's `ExternalHandler.makeExternalCalls()` is completed but the `refundReceivers` do not receive any USDT/GMX as they were stolen by Alice.

On the other hand, the following exploitation path is also present:

M-02 ExternalHandler Tokens Could Be Drained

CONTINUED

1. Alice calls the `ExternalHandler.makeExternalCalls()` passing as parameter the following external call `RewardRouterV2.signalTransfer(<Alice's smart contract>)`. This external call does the following update: `pendingReceivers[<ExternalHandler>] = <Alice's smart contract>;`
2. Bob sends `exGMX` and `bnGmx` to the `ExternalHandler` contract and then perform multiple external calls. One of these external calls gives the control to Alice's smart contract which calls `RewardRouterV2.acceptTransfer(<ExternalHandler>):`.
3. Alice's smart contract drains, this way, all the `exGMX` and `bnGmx` tokens from the `ExternalHandler` contract.

This attack scenario may be made more likely if tokens with callbacks are used in the external actions.

— RECOMMENDATION

Consider adding an extra parameter to the `ExternalHandler.makeExternalCalls()` function which specifies the `minAmountOut` that should be refunded. Any call to this function that does not refund more tokens than the `minAmountOut` to the `refundReceivers` should revert.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

esGMX Claimed To Any Address

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Currently the `stakedGmxTracker` has `privateClaimingMode` set to false, therefore the reward token, esGMX, can be claimed to any receiver address.

This effectively bypasses the non-transferrable nature of esGMX and allows users to leverage the `isHandler` privilege of the `RewardTracker` contract to send their esGMX rewards to any address they would like.

This enables issues like H-03 even after the esGMX emissions have been disabled since some accounts may have not yet claimed their esGMX rewards.

— RECOMMENDATION

Update the configuration of the `stakedGmxTracker` contract such that `privateClaimingMode` is true.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Users Cannot Unstake Until Re-staked

CATEGORY	Documentation
LOCATION	RewardRouterV2.sol: 541
REVIEW	Main Review

— DESCRIPTION

In the current deployment plan for the new RewardRouterV2 contract it is mentioned that the order of operations will be as follows:

- deploy new RewardRouterV2
- enable new RewardRouterV2
- have interface use new RewardRouterV2
- notify integrations to use new RewardRouterV2
- restake for all accounts
- disable old RewardRouterV2

However once the UI is updated to use the new RewardRouterV2 contract users will be unable to unstakeGmx, unstakeEsGmx, compound, claim, claimFees until they are re-staked through the extendedGmxTracker.

This may be unexpected, especially as it may take a non-trivial amount of time to re-stake all accounts.

— RECOMMENDATION

This issue cannot be solved by re-ordering the deployment plan, as the same issue would persist for users who were already re-staked, and trying to interact with the old RewardRouterV2 contract.

Instead, be sure to document this behavior for users so that this does not come as a surprise.

— RESOLUTION

Status: Resolved.

LOW **L-02**

ACKNOWLEDGED

RewardRouterV2 Contract Missing GOV_TOKEN_CONTROLLER Role

CATEGORY DoS
LOCATION RewardRouterV2.sol
REVIEW Main Review

DESCRIPTION

The `RewardRouterV2` contract implements the internal function `_syncVotingPower`:

```
function _syncVotingPower(address _account, uint256 _amount) private {  
    uint256 currentVotingPower = IERC20(govToken).balanceOf(_account);  
    if (currentVotingPower == _amount) { return; }  
    if (currentVotingPower > _amount) {  
        uint256 amountToBurn = currentVotingPower.sub(_amount);  
        IMintable(govToken).burn(_account, amountToBurn);  
    }  
    return;  
}  
  
uint256 amountToMint = _amount.sub(currentVotingPower);  
IMintable(govToken).mint(_account, amountToMint);  
}
```

This function ensures that a user's governance token balance (voting power) is updated after actions such as staking or unstaking. If the user's current voting power exceeds the target amount (`_amount`), the excess tokens are burned. Conversely, if the voting power is lower than the target, the difference is minted to the user's account, thereby synchronizing the governance token balance with the required voting power.

In order to be able to mint/burn the governance tokens, the `RewardRouterV2` contract must be given the `GOV_TOKEN_CONTROLLER` role in the `RoleStore` contract [<https://arbiscan.io/address/0x3c3d99FD298f679DBC2CEcd132b4eC4d0F5e6e72#code>].

RECOMMENDATION

Ensure that the new `RewardRouterV2` contract is given the `GOV_TOKEN_CONTROLLER` role in the `RoleStore` contract.

L-02 RewardRouterV2 Contract Missing GOV_TOKEN_CONTROLLER RoleCONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

RESOLVED

_restakeBnGmx Ignores RewardRouterV2 maxAllowedBnGmxAmount

CATEGORY Logical Error
LOCATION RewardRouterV2.sol: 645
REVIEW Main Review

— DESCRIPTION

In the `RewardRouterV2` contract the `maxBoostBasisPoints` state variable determines the maximum amount of bnGMX tokens that can be staked into the `ExtendedGmxTracker` contract.

Additionally, the `RewardRouterV2` contract implements the function `batchRestakeForAccounts()`:

```
function batchRestakeForAccounts(address[] memory _accounts) external nonReentrant onlyGov {
    for (uint256 i = 0; i < _accounts.length; i++) {
        _restakeForAccount(_accounts[i]);
    }
}

function _restakeForAccount(address _account) private {
    uint256 bonusGmxTrackerBalance = IRewardTracker(feeGmxTracker).depositBalances(_account,
    bonusGmxTracker);
    if (bonusGmxTrackerBalance > 0) {
        uint256 reservedForVesting = IVester(gmxVester).pairAmounts(_account);
        if (reservedForVesting > 0) {
            IERC20(feeGmxTracker).safeTransferFrom(gmxVester, _account, reservedForVesting);
            _restakeBonusGmxTracker(_account, bonusGmxTrackerBalance);
            _restakeBnGmx(_account);
            IERC20(feeGmxTracker).safeTransferFrom(_account, gmxVester, reservedForVesting);
        }
        else {
            _restakeBonusGmxTracker(_account, bonusGmxTrackerBalance);
            _restakeBnGmx(_account);
        }
    }
}
```

L-03 _restakeBnGmx Ignores RewardRouterV2 maxAllowedBnGmxAmount

CONTINUED

```
function _restakeBnGmx(address _account) private {  
    uint256 stakedBnGmx = IRewardTracker(feeGmxTracker).depositBalances(_account, bnGmx);  
    if (stakedBnGmx > 0) {  
        IRewardTracker(feeGmxTracker).unstakeForAccount(_account, bnGmx, stakedBnGmx, _account);  
        IRewardTracker(extendedGmxTracker).stakeForAccount(_account, _account, bnGmx, stakedBnGmx);  
        IRewardTracker(feeGmxTracker).stakeForAccount(_account, _account, extendedGmxTracker,  
stakedBnGmx);  
    }  
}
```

This function is used to move stakes from the old router to the new one. However, the re-staking process does not verify if the bnGMX tokens staked in the `ExtendedGmxTracker` exceed the limit imposed by `maxBoostBasisPoints`. This lack of a limit check could allow accounts to bypass the staking cap for bnGMX tokens on the `ExtendedGmxTracker`.

Finally, there is a nonzero amount of bnGMX in the balance of the sbfGMX contract address. There is no trivial way to check if this is currently deposited on behalf of a user or merely sitting in the contract balance. But if the former is the case then that user's bnGMX would be restaked through the `extendedGmxTracker`, ignoring the `maxBoostBasisPoints` which may lead to unexpected behaviors.

— RECOMMENDATION

Consider implementing a check within the re-staking process. Specifically, before staking bnGMX tokens in the `ExtendedGmxTracker`, a condition should be added to ensure that the total staked bnGMX for an account does not exceed the limit defined by `maxBoostBasisPoints`.

Additionally, consider verifying whether the bnGMX sitting in the sbfGmx contract address belongs to a user's deposited amount. If this is the case, consider burning it from that user.

— RESOLUTION

Status: Resolved.

LOW **L-04**

ACKNOWLEDGED

FeeGmxTracker Should Only Allow To Stake sbeGMX

CATEGORY Configuration
 LOCATION FeeGmxTracker.sol
 REVIEW Main Review

DESCRIPTION

The table below represents the staking/rewards setup after the addition of the new `ExtendedGmxTracker` contract. This is the logic that will be followed as well by the new `RewardRouterV2` contract:

Tracker	Stakes	Rewards
StakedGMXTracker (sGMX)	GMX, esGMX	esGMX
BonusGmxTracker (sbGMX)	sGMX	bnGMX
ExtendedGmxTracker (sbeGMX)	sbGMX, bnGMX	GMX
FeeGmxTracker (sbfGMX)	sbeGMX	WETH
FeeGlpTracker (fGLP)	GLP	WETH
StakedGLPTracker (fsGLP)	fGLP	esGMX
GMXVester	esGMX	GMX
GLPVester	esGMX	GMX

Consequently, as the `FeeGmxTracker` currently supports sbGMX and bnGMX deposits, these depositTokens should be removed from the `FeeGmxTracker` after the upgrade to the new `RewardRouterV2` is completed.

RECOMMENDATION

Ensure that sbGMX and bnGMX are disabled as depositTokens in the `FeeGmxTracker` when the upgrade is completed.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Multiple Operations Require An Unintended GMX/bnGMX Approval

CATEGORY Configuration
LOCATION RewardRouterV2: 471, RewardRouterV2: 487
REVIEW Main Review

DESCRIPTION

The `RewardRouterV2` contract implements the `compound` function:

```
function compound() external nonReentrant {
    _compound(msg.sender);
}

function _compound(address _account) private {
    uint256 gmxAmount = _claimGmxFees(_account, _account);
    if (gmxAmount > 0) {
        _stakeGmx(_account, _account, gmx, gmxAmount);
    }
    uint256 esGmxAmount = _claim(stakedGmxTracker, stakedGlpTracker, _account, _account);
    if (esGmxAmount > 0) {
        _stakeGmx(_account, _account, esGmx, esGmxAmount);
    }
    _stakeBnGmx(_account);
    _syncVotingPower(_account);
}
```

This function stakes the GMX/bnGMX tokens by calling the `_stakeGmx` internal function:

```
function _stakeGmx(address _fundingAccount, address _account, address _token, uint256 _amount)
private {
    _validateAmount(_amount);
    IRewardTracker(stakedGmxTracker).stakeForAccount(_fundingAccount, _account, _token, _amount);
    IRewardTracker(bonusGmxTracker).stakeForAccount(_account, _account, stakedGmxTracker,
    _amount);
    IRewardTracker(extendedGmxTracker).stakeForAccount(_account, _account, bonusGmxTracker,
    _amount);
    IRewardTracker(feeGmxTracker).stakeForAccount(_account, _account, extendedGmxTracker,
```

L-05 Multiple Operations Require An Unintended GMX/bnGMX Approval

CONTINUED

```
_amount);  
_syncVotingPower(_account);  
emit StakeGmx(_account, _token, _amount);  
}`
```

The `StakedGmxTracker` will pull the GMX tokens from the `_fundingAccount` and stake them into the contract. This is the `transferFrom` implementation of the GMX/bnGMX tokens:

```
function transferFrom(address _sender, address _recipient, uint256 _amount) external override  
returns (bool) {  
    if (isHandler[msg.sender]) {  
        _transfer(_sender, _recipient, _amount);  
        return true;  
    }  
    uint256 nextAllowance = allowances[_sender][msg.sender].sub(_amount, "BaseToken: transfer  
amount exceeds allowance");  
    _approve(_sender, msg.sender, nextAllowance);  
    _transfer(_sender, _recipient, _amount);  
    return true;  
}
```

However, as the `StakedGmxTracker` contract is not a handler for the GMX token, the contract can not transfer tokens from an account without enough allowance. (Currently esGMX has already set as a handler the `StakedGmxTracker` contract).

Therefore, users would be required to GMX approve the `StakedGmxTracker` contract before any compound or stake call. This step would not be necessary if the `StakedGmxTracker` contract was a handler for the GMX contract. This addition should be safe as the `stakeGmx`, `stakeEsGmx`, `compound` and `handleRewards` functions directly operate with the `caller/msg.sender` account.

Additionally, the `ExtendedGmxTracker` tries to pull the bnGMX tokens from the `_fundingAccount` and stake them into the `ExtendedGmxTracker` contract, and thus, bnGMX should also set the `ExtendedGmxTracker` contract as a handler.

The current impact is that all the operations below would require 2 extra approvals from the caller, 1 for GMX and 1 for bnGMX:

L-05 Multiple Operations Require An Unintended GMX/bnGMX Approval**CONTINUED**

- `stakeGMX`
- `compound`
- `handleRewards`
- `handleRewardsV2`
- `acceptTransfer`
- `batchCompoundForAccounts`

RECOMMENDATION

Consider adding the `StakedGmxTracker` contract as a handler for GMX. Moreover, consider also adding the `ExtendedGmxTracker` as a handler of the bnGMX contract. By doing so, the contracts would have the necessary permissions to transfer tokens on behalf of users without requiring manual approval for each transaction. This would streamline the user experience by eliminating the need for separate `approve` calls before staking, handling rewards and compounding GMX/bnGMX tokens.

RESOLUTION

Status: Resolved.

LOW **L-06**

ACKNOWLEDGED

Missing Initialized Values

CATEGORY	Warning
LOCATION	RewardRouterV2.sol: 130
REVIEW	Main Review

DESCRIPTION

The following variables are not assigned with an initial value in the initialize function:

- maxBoostBasisPoints
- inStrictTransferMode
- votingPowerType

RECOMMENDATION

Consider if these values should be able to have a nonzero initial value upon initialization and implement these initial assignments in the initialize function if necessary.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Lacking Zero Address Checks

CATEGORY	Validation
LOCATION	RewardRouterV2.sol
REVIEW	Main Review

DESCRIPTION

In the RewardRouterV2 contract there are several functions which configure addresses which do not validate against the zero address input.

RECOMMENDATION

Consider adding zero address checks to the initialize and setGovToken functions.

RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

VesterCap May Not Function Correctly

CATEGORY Warning
LOCATION VesterCap.sol
REVIEW Main Review

— DESCRIPTION

The VesterCap._updateBnGmxForAccount function assumes that accounts have been re-staked as it attempts to unstake amounts from the extendedGmxTracker.

Thus some accounts may be immune to the VesterCap actions if they leveraged any of the methods to avoid the re-staking which we highlighted in other findings.

The VesterCap is not planned for future use, this finding serves only to document this risk.

— RECOMMENDATION

Be aware of this risk and consider it for future VesterCap usage.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

VesterCap maxBoostBasisPoints Inconsistency

CATEGORY	Warning
LOCATION	VesterCap.sol
REVIEW	Main Review

— DESCRIPTION

The VesterCap contract holds a maxBoostBasisPoints storage variable which could be inconsistent with the value used in the RewardRouterV2 contracts.

— RECOMMENDATION

Consider verifying that all live VesterCap contracts have the same maxBoostBasisPoints as the RewardRouterV2 contracts. Additionally if future use is desired, consider refactoring the VesterCap to rely on the maxBoostBasisPoints stored in the relevant RewardRouterV2 contract.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

RESOLVED

Missing nonReentrant Modifier

CATEGORY	Reentrancy
LOCATION	RewardRouterV2.sol: 180
REVIEW	Main Review

DESCRIPTION

The makeExternalCalls function in the RewardRouterV2 contract does not have a nonReentrant modifier.

RECOMMENDATION

Consider adding a nonReentrant modifier to this function.

RESOLUTION

Status: Resolved.

LOW **L-11**

RESOLVED

validateReceiver Is Missing Checks On The extendedGmxTracker

CATEGORY Logical Error
LOCATION RewardRouterV2.sol: 654
REVIEW Main Review

DESCRIPTION

The `RewardRouterV2` contract implements the function `validateReceiver()`:

```
function _validateReceiver(address _receiver) private view {  
    require(IRewardTracker(stakedGmxTracker).averageStakedAmounts(_receiver) == 0,  
        "stakedGmxTracker.averageStakedAmounts > 0");  
    require(IRewardTracker(stakedGmxTracker).cumulativeRewards(_receiver) == 0,  
        "stakedGmxTracker.cumulativeRewards > 0");  
    require(IRewardTracker(bonusGmxTracker).averageStakedAmounts(_receiver) == 0,  
        "bonusGmxTracker.averageStakedAmounts > 0");  
    require(IRewardTracker(bonusGmxTracker).cumulativeRewards(_receiver) == 0,  
        "bonusGmxTracker.cumulativeRewards > 0");  
    require(IRewardTracker(feeGmxTracker).averageStakedAmounts(_receiver) == 0,  
        "feeGmxTracker.averageStakedAmounts > 0");  
    require(IRewardTracker(feeGmxTracker).cumulativeRewards(_receiver) == 0,  
        "feeGmxTracker.cumulativeRewards > 0");  
    require(IVester(gmxVester).transferredAverageStakedAmounts(_receiver) == 0,  
        "gmxVester.transferredAverageStakedAmounts > 0");  
    require(IVester(gmxVester).transferredCumulativeRewards(_receiver) == 0,  
        "gmxVester.transferredCumulativeRewards > 0");  
    require(IRewardTracker(stakedGlpTracker).averageStakedAmounts(_receiver) == 0,  
        "stakedGlpTracker.averageStakedAmounts > 0");  
    require(IRewardTracker(stakedGlpTracker).cumulativeRewards(_receiver) == 0,  
        "stakedGlpTracker.cumulativeRewards > 0");  
    require(IRewardTracker(feeGlpTracker).averageStakedAmounts(_receiver) == 0,  
        "feeGlpTracker.averageStakedAmounts > 0");  
    require(IRewardTracker(feeGlpTracker).cumulativeRewards(_receiver) == 0,  
        "feeGlpTracker.cumulativeRewards > 0");  
    require(IVester(glpVester).transferredAverageStakedAmounts(_receiver) == 0,  
        "gmxVester.transferredAverageStakedAmounts > 0");
```

L-11 validateReceiver Is Missing Checks On The extendedGmxTracker

CONTINUED

```
require(IVester(glpVester).transferredCumulativeRewards(_receiver) == 0,
"gmxBVester.transferredCumulativeRewards > 0");
require(IERC20(gmxBVester).balanceOf(_receiver) == 0, "gmxBVester.balance > 0");
require(IERC20(glpVester).balanceOf(_receiver) == 0, "glpVester.balance > 0");
}
```

This function is used in the `signal/acceptTransfer()` logic, to check that the `averageStakedAmounts` and `cumulativeRewards` values of an account are zero ensuring that vesting calculations are executed correctly.

After this update, the `ExtendedGmxTracker` would also be used as part of the `stakeGMX()` flow. The `RewardRouterV2` would stake sbGMX and bnGMX tokens to receive GMX tokens as rewards. However, the `_validateReceiver()` function fails to check that the `averageStakedAmounts` and `cumulativeRewards` of this new reward tracker are zero.

However there is no identified impact of this issue as bnGMX cannot be staked currently so it isn't possible to achieve an account with only a nonzero `averageStakedAmount` in the `extendedGmxTracker` and a zero `averageStakedAmount` for all other `RewardTracker` contracts.

— RECOMMENDATION

Update the `_validateReceiver()` function so it enforces that the `averageStakedAmounts` and `cumulativeRewards` of the `ExtendedGmxTracker` are zero.

— RESOLUTION

Status: Resolved.

LOW **L-12**

ACKNOWLEDGED

Some Accounts Cannot Be Re-Staked

CATEGORY	Warning
LOCATION	RewardRouterV2.sol
REVIEW	Main Review

— DESCRIPTION

In the `_restakeForAccount` function only accounts where the deposited `bonusGmxTracker` amount in the `feeGmxTracker` contract is nonzero can be restaked.

However for accounts where there is only bnGMX staked in the `feeGmxTracker`, these accounts will not be able to be processed.

This is not a large issue as no accounts should have any staked bnGMX at this point, but it is worth noting.

— RECOMMENDATION

Be aware of this issue, and confirm that no accounts have deposited amounts for bnGMX on both Arbitrum and Avalanche.

— RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Some Accounts May Need To Be ReReStaked

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

While both RewardRouterV2 contracts are live, it is possible that a user can be re-staked through the new system and then manually unstake and stake through the old RewardRouterV2 contract.

In this case the same user would have to be re-re-staked for a second time. This may also occur if the user calls the compound or handleRewards function on the old RewardRouterV2 contract.

— RECOMMENDATION

Be aware of this possibility and be prepared to re-stake users for a second time after the old reward router has been disabled.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Private Staking Mode

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review

DESCRIPTION

This finding simply serves as a warning/reminder to be sure to configure the `extendedGmxTracker` to have private staking mode as true.

Otherwise unexpected behavior may occur as users can stake through it before the new reward router is enabled with the migrator. This is because the deposit tokens will be set upon initialization of the `extendedGmxTracker`.

RECOMMENDATION

Be sure to set `inPrivateStakingMode` as true when deploying the `extendedGmxTracker`.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

Unused Variable

CATEGORY	Superfluous Code
LOCATION	RewardRouterV2.sol
REVIEW	Remediation Review

DESCRIPTION

In the `RewardRouterV2` contract a `inRestakingMode` variable has been introduced with a corresponding setter.

However this variable is not used for anything notable.

RECOMMENDATION

Consider either implementing it's usecase or removing the `inRestakingMode` variable.

RESOLUTION

Status: Resolved.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.